

FinGuard Orchestrator: Enterprise Agentic Wealth Management Compliance & AML Audit Engine

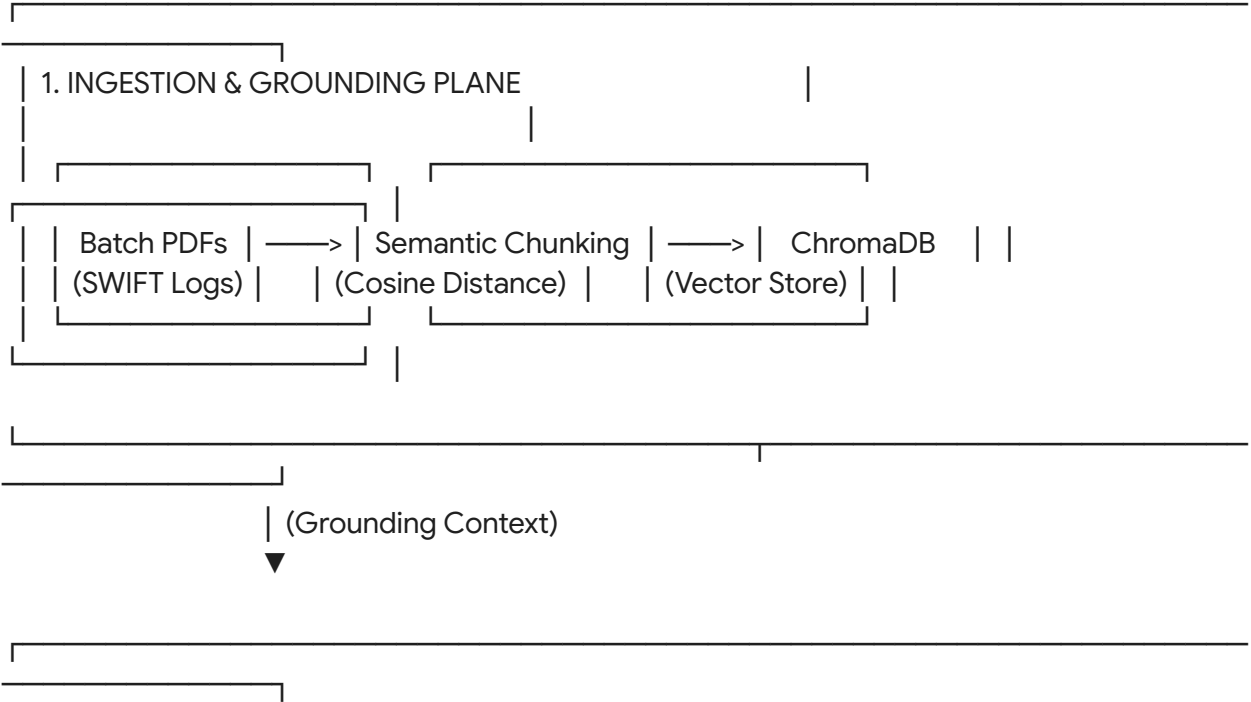
1. Executive Summary & Project Vision

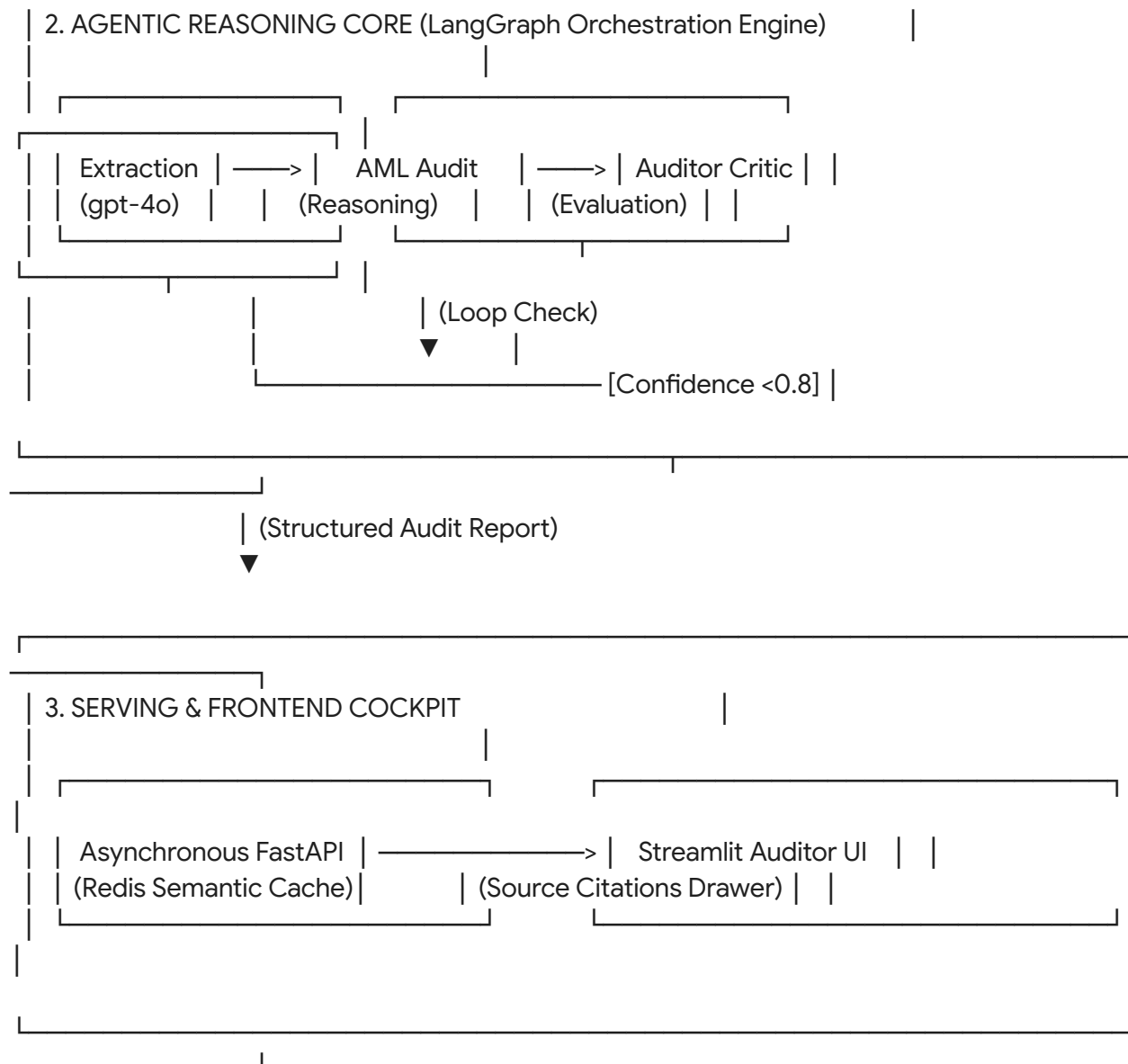
FinGuard Orchestrator is an enterprise-grade AI system designed to solve a massive operational friction point in the financial sector: **Compliance and Anti-Money Laundering (AML) transaction auditing**. Traditional compliance tracking struggles with unstructured text data—such as wire transfer memo lines, unstructured payment descriptions, wealth portfolio disclosures, and rapid international regulatory updates.

This project builds an autonomous, multi-agent auditor that ingests batch transaction logs, parses transactional structures into standardized entities, audits them against a dynamic regulatory compliance database, evaluates transaction runs for suspicious patterns, and outputs structured Suspicious Activity Reports (SARs) backed by verifiable legal citations.

2. Architectural Overview & Data Flow

The system is decoupled into three primary functional zones to guarantee enterprise reliability and horizontal scalability:





1. **The Ingestion & Semantic Grounding Plane:** Parses complex PDFs of financial standards and transactional logs, splits them via embedding similarity metrics to preserve contextual meaning, and hosts them in a vector database.
2. **The Agentic Core (LangGraph Engine):** A stateful directed graph consisting of specialized extraction, analytical auditing, and critique nodes designed to run loops until compliance assertions are resolved.
3. **The Serving & Frontend Cockpit:** An asynchronous FastAPI microservice backed by a Streamlit interactive frontend, monitored end-to-end via LangSmith or Logfire.

3. Phase 1: Ingestion & Semantic Grounding (The "Data

Body")

Instead of simple character-based splits that fracture critical ledger records or legal guidelines, we implement **Semantic Chunking**.

3.1 High-Performance Environment Setup

We initialize our workspace using **uv** for lightning-fast, deterministic dependency resolution:

```
# Initialize workspace
uv init finguard-orchestrator
cd finguard-orchestrator
uv venv
source .venv/bin/activate
```

```
# Add core engineering dependencies
```

```
uv add langchain langchain-openai langchain-community streamlit chromadb pypdf pydantic
redis fastapi uvicorn deeveval ragas
```

3.2 Real-World Dataset Strategy & Acquisition

To establish maximum professional credibility, we ground our platform on two authentic, production-grade financial corpora:

A. Transaction & Payment Ledger Feed (The Auditor Input)

We utilize the **SAML-D (Synthetic Anti-Money Laundering Dataset)**, a high-fidelity transactional dataset designed specifically to train and evaluate financial transaction monitoring systems.

- **Database Target:** Tracks features including Unique Account IDs, transfer corridors, UTC timestamps, geographic country codes, payment volume, and binary labels indicating suspicious behavior.
- **Money Laundering Typologies Covered:** Structuring (breaking down a large sum of money into several transfers below compliance reporting thresholds), circular laundering cycles, scatter-gather movements, and tax-haven routing.
- **SWIFT ISO 20022/pacs.008 Mapping:** We provide a Python script at `src/utils/pdf_generator.py` that reads these tabular transactional runs and formats them into raw SWIFT network messages containing:
 - **Block 1 & 2 Headers:** Sender/Receiver Bank Identifier Codes (BICs).
 - **Block 4 Application Data:** Field :20: Transaction Reference (UETR), Field :32A: Value Date & Amount, and Field :50a: Ordering Customer.
 - **Output Container:** These records are rendered as simulated "Monthly Private Banking Institutional Transaction Logs" inside unformatted text or PDF statement files for

uploader ingestion.

- **Acquisition Method:** Download directly from Kaggle: [Synthetic Transaction Monitoring Dataset \(SAML-D\)](#).

B. Dynamic Regulatory Compliance Rules (The Knowledge Base)

We ground our vector store using a hybrid legal corpus composed of:

1. **The ObliQA Financial Regulatory Dataset:** A curated legal NLP corpus containing dense Abu Dhabi Global Market (ADGM) compliance guidelines. This data maintains intricate legal layouts (section numbering, deeply nested lists, and bullet points) that test the limits of semantic chunking parsing. Download from: [RegNLP/ObliQADataset](#).
 2. **Authoritative Anti-Money Laundering Rulebooks:** Raw PDF versions of official US regulatory guidance documents, specifically:
 - **FINRA Rule 3310 (AML Compliance Program)** and **Rule 3110 (Supervision)**.
 - **FinCEN Advisory Notices** detailing active red-flag indicators for geographic tax havens, shell company structuring, and illicit money movement corridors.
- **Why this is used:** Parsing official PDFs with multi-column formats, headers, and legal footnotes forces our semantic parsing logic to be exceptionally resilient.

3.3 Semantic Chunking Logic

We split unstructured document feeds by analyzing the cosine distance between adjacent sentence embeddings. If the similarity drops below a calculated threshold, we create a new chunk boundary, keeping coherent legal and financial entities together.

The mathematical representation of sentence similarity is defined as:

$$\text{Similarity}(S_i, S_{i+1}) = \frac{\vec{S}_i \cdot \vec{S}_{i+1}}{\|\vec{S}_i\| \|\vec{S}_{i+1}\|}$$

3.4 Ingestion Pipeline Milestones

- Build parsing scripts inside `src/ingestion/loader.py` to process unstructured compliance rules and ledger transactions.
- Initialize local ChromaDB vector storage, injecting metadata vectors (`source_file`, `section_clause`, `last_updated_date`) to ensure accurate source attribution.
- Implement the initial Streamlit interface indicating vector store ingestion metrics.

4. Phase 2: Stateful Multi-Agent Graph (The "Brain")

Rather than writing linear request-response API calls, we orchestrate our system as a stateful graph using **LangGraph**.

4.1 Graph State Definition

The shared memory object passing between our execution nodes is defined using Pydantic typing:

```
from typing import List, TypedDict, Dict, Any
from langchain_core.documents import Document
```

```
class AgentState(TypedDict):
    raw_query: str
    extracted_entities: Dict[str, Any]
    retrieved_context: List[Document]
    compliance_draft: str
    loop_count: int
    confidence_score: float
    is_audit_complete: bool
```

4.2 Graph Node Topology

1. **Extraction Node:** Uses a fast model (e.g., gpt-4o-mini) to parse transactional raw text logs, identifying and standardizing sender accounts, payment amounts, SWIFT/ACH routing codes, and country corridors.
2. **AML Audit Node:** Queries ChromaDB for legal guidelines associated with extracted entities, matching indicators against known patterns (such as structuring or money routing to high-risk geographical zones).
3. **Auditor Critic Node:** Audits the draft report for factual accuracy. If confidence is below 0.80 and loop iterations are below 3, it triggers query refinement and reroutes execution back to search again.
4. **Generation Node:** Formulates the finalized Suspicious Activity Report (SAR) in structural markdown with strict citation hashes.

5. Phase 3: Structured Output & Citations (The "Enterprise Interface")

In enterprise banking, compliance records must be structured, deterministic, and verifiable.

5.1 Pydantic Schema Enforcement

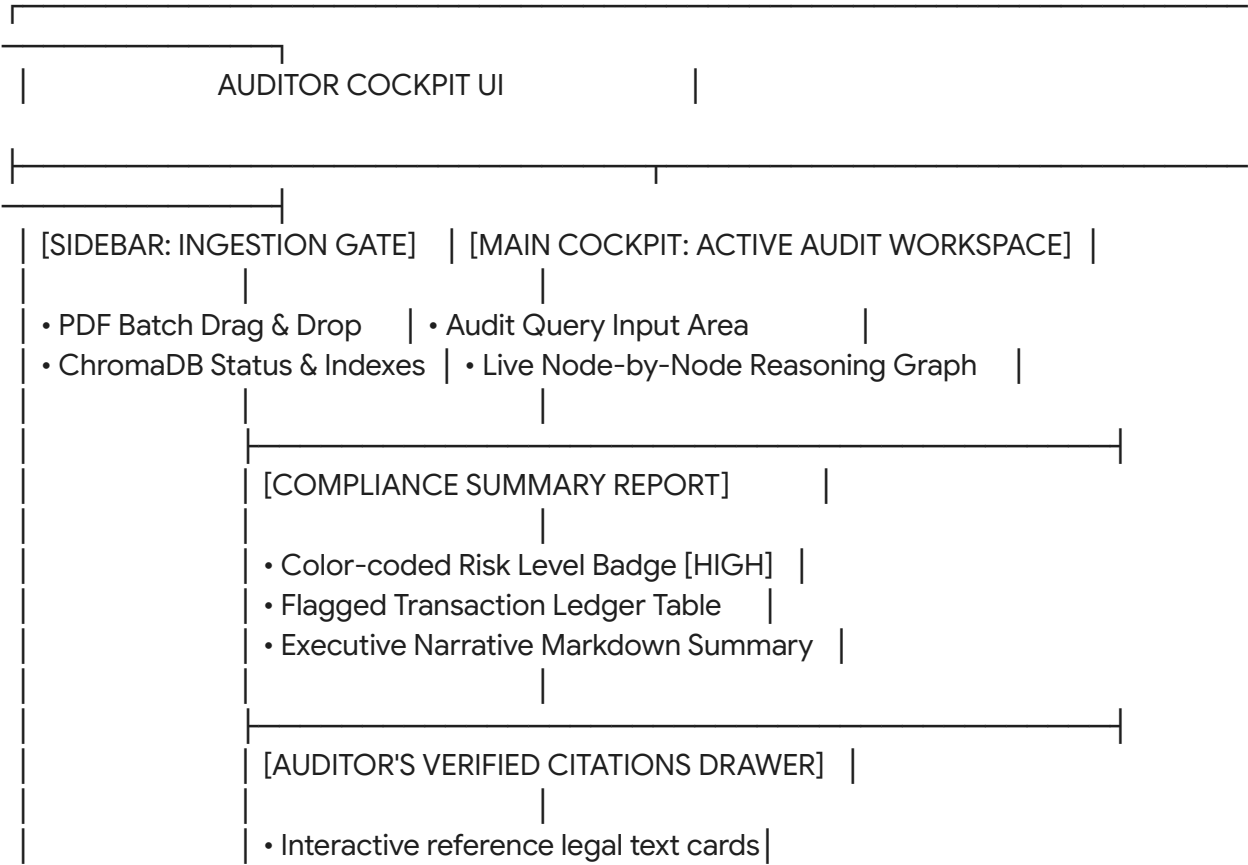
We bind structured schemas to the generation layer to guarantee that the agent always outputs standard JSON formats rather than loose, non-deterministic text.

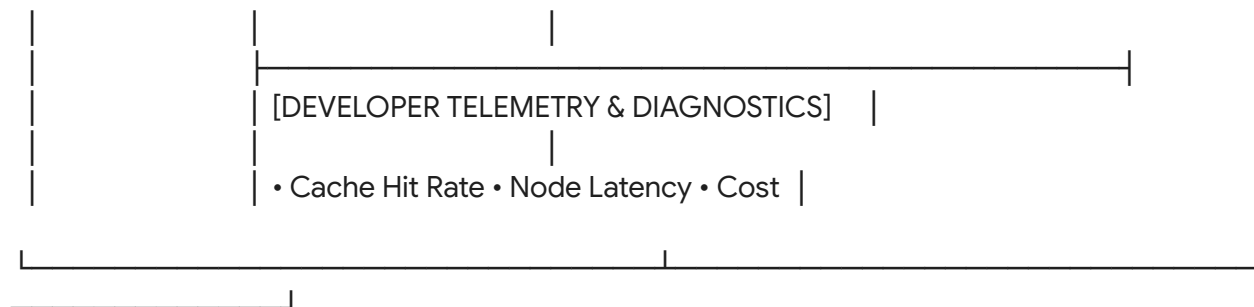
```
from pydantic import BaseModel, Field
from typing import List
```

```
class ComplianceReport(BaseModel):
    risk_rating: str = Field(description="Low, Medium, or High Risk Assessment of transaction run")
    flagged_wires: List[str] = Field(description="List of suspicious wire reference IDs matching illegal AML patterns")
    applicable_regulations: List[str] = Field(description="References to audited compliance sections and regulatory clauses")
    audit_summary: str = Field(description="Markdown formatted detailed explanation of the analytical findings")
    source_document_hashes: List[str] = Field(description="Cryptographic or metadata IDs of cited source compliance records")
```

6. Phase 4: Streamlit UI & Auditor Cockpit

The user experience is designed as an interactive, live audit interface. The frontend is divided into five core functional sections to make the AI's internal processes fully transparent and observable:





6.1 The Ingestion Sidebar (The Data Gate)

- **Document Uploader:** Drag-and-drop landing target for PDF transactions (SAML-D converted SWIFT message blocks) or regulatory updates (ObliQA, FINRA, FinCEN advisories).
- **Indexing Monitor:** A visual progress bar detailing parsing status, sentence similarity boundaries, and vectors written to ChromaDB.
- **Vector Store Inventory:** Shows a list of loaded regulations (e.g., *FINRA Rule 3310*, *ADGM AML Rulebook*) so the auditor knows which rules are active.

6.2 The Active Audit Workspace (The Execution View)

- **Command Bar:** Auditor inputs instructions, e.g., *"Audit current ledger for structuring maneuvers below \$10,000 limits."*
- **Reasoning Graph Tracker:** Employs Streamlit status indicators to visualize which node in the LangGraph DAG is actively processing:
 -  *Step 1: Normalizing SWIFT transactional data via Extraction Node (gpt-4o-mini)...*
 -  *Step 2: Cross-referencing entities against ChromaDB via AML Audit Node...*
 -  *Step 3: Calculating compliance score via Auditor Critic Node...*
 -  *Step 4: Compiling finalized compliance draft...*

6.3 The Compliance Summary Report (The Decision Center)

- **Risk Badge:** Highly visible, color-coded status banners displaying

HIGHRISK – RED

,

MEDIUMRISK – YELLOW

, or

- **Flagged Transaction Ledger:** Rendered using dataframes (st.dataframe) detailing transaction IDs, sender/recipient accounts, and transaction volumes for quick auditing.
- **Executive Narrative:** Styled markdown detail of legal and AML operational violations.

6.4 The Auditor's Citations Drawer (The Audit Trail)

To eliminate hallucinations, an interactive drawer element at the bottom parses the output's source_document_hashes and links them directly to the source text strings stored inside ChromaDB, showing the exact legal clause used to justify each flag.

6.5 The "Telemetry & Diagnostics" Panel (The Engineering Showcase)

A togglable developer diagnostics dashboard designed to showcase system design maturity:

- **Semantic Cache Monitor:** Displays CACHE HIT  or CACHE MISS  depending on Redis state matching.
- **Execution Latency Breakdown:** Real-time metrics detailing millisecond speeds per LangGraph node.
- **Pricing Calculator:** Track token throughput and precise API cost metrics per query.

7. Phase 5: Production Observability (LangSmith Integration)

In a production environment, traditional log output is completely blind to agentic reasoning. We implement complete **LangSmith** tracing to provide deep visual observability into the state graph's trace executions.

7.1 Tracing Configuration

We export tracing variables directly in our runtime environments to log pipeline actions:

```
export LANGCHAIN_TRACING_V2="true"
export LANGCHAIN_API_KEY="your-langsmith-api-key"
export LANGCHAIN_PROJECT="finguard-orchestrator"
```

7.2 Deep Tracing Custom Metadata & Run-Tagging

To easily slice and debug transactional pipelines inside the LangSmith platform, we inject targeted corporate meta-arguments into the state invocations:


```
# Invoke LangGraph using scoped run configuration for fine-grained filters
config = {
    "tags": ["AML_AUDIT_RUN", "BATCH_PROD"],
    "metadata": {
        "client_tier": "VIP_Institutional",
        "audit_id": "aud-9988-xx",
        "batch_wire_count": len(state["extracted_entities"].get("wires", [])),
    }
}

# Invoke asynchronously to keep event loops unblocked
await graph.ainvoke(initial_state, config=config)
```

By adding these metadata keys, Bill will be able to search LangSmith for specific traces of VIP transactions, compare prompt latency variations across different models, and pinpoint precisely which context block caused the **Auditor Critic** to trigger a loop revision.

8. Phase 6: Automated LLM Evaluations (DeepEval & RAGAS Framework)

Traditional software engineering relies on assertions and static Unit Tests. LLMs require **probabilistic validation** using an LLM-as-a-judge framework to ensure prompts and retrieval layers don't degrade over time.

We implement an offline evaluation suite at tests/eval_suite.py using **DeepEval** to test the pipeline against a "Gold Standard" test set containing 15 complex AML mock compliance scenarios.

8.1 Evaluating Key Performance Metrics

We evaluate three core metrics essential for financial compliance auditing:

1. **Faithfulness (Hallucination Detection):** Verifies that the produced compliance summary derives *only* from the retrieved context sections inside ChromaDB.

$$\text{Faithfulness} = \frac{\text{Number of Claims Grounded in Context}}{\text{Total Claims Made in Report}}$$

2. **Answer Relevancy:** Measures how directly the generated report addresses the auditor's initial query request.
3. **Context Precision:** Ensures that the semantic chunking retrieve node ranks the actual

legally relevant FINRA clauses or ADGM sections in the high K -ranking parameters.

8.2 Automated Eval Harness Implementation

We write a test executable using DeepEval to run automatically as a quality gate inside Bill's GitHub Actions deployment pipeline:

```
# File: tests/eval_suite.py
import pytest
from deeveval.metrics import FaithfulnessMetric, AnswerRelevancyMetric
from deeveval.test_case import LLMTestCase
from deeveval import assert_test

def test_aml_generation_quality():
    # 1. Define a golden test case
    test_case = LLMTestCase(
        input="Analyze transaction references TXN-093 for structuring compliance violations.",
        actual_output="Transaction TXN-093 splits a $34,000 transfer into four distinct $8,500 batches over 48 hours to evade the $10,000 Currency Transaction Reporting requirements. This violates FINRA Rule 3310(a).",
        retrieved_context=[
            "FINRA Rule 3310(a) mandates Member firms must implement transactional monitoring designed to detect structuring, defined as dividing single transactions into consecutive segments below $10,000 reporting thresholds."
        ]
    )

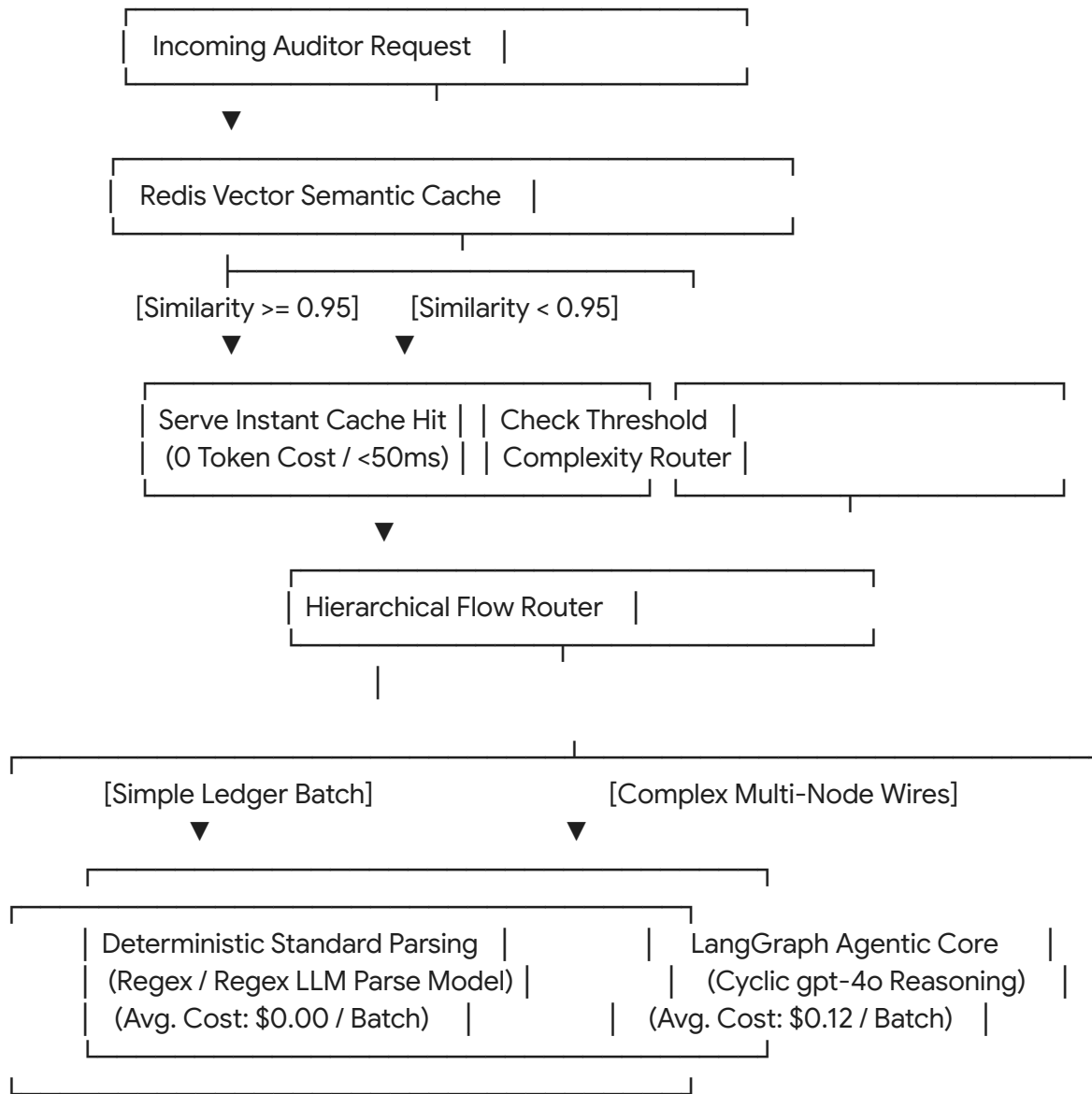
    # 2. Instantiate metrics with target thresholds
    faithfulness_metric = FaithfulnessMetric(threshold=0.85, model="gpt-4o")
    relevancy_metric = AnswerRelevancyMetric(threshold=0.80, model="gpt-4o")

    # 3. Assert metrics to validate alignment
    assert_test(test_case, [faithfulness_metric, relevancy_metric])
```

9. Phase 7: Advanced Cost & Latency Optimization Architecture

Running stateful graphs over expensive models (like gpt-4o) can quickly run up high API costs. We design an enterprise cost optimization topology combining **Hierarchical Routing**, **Semantic Cache**, and **Context Pruning**.

9.1 Cost Optimization Topology



9.2 Hierarchical Graph Routing (Determining Graph Escalation)

Not all ledgers require complex, recursive multi-agent auditing. Bill will implement a lightweight **Rule-Based Pre-Router Node** using a regex signature scanner or a tiny local classification model:

- **The Logic:** If the incoming log contains simple, low-volume local ACH payments with clean memo notes and values under **\$3,000**, it completely bypasses the LangGraph agent

core. It processes immediately using a deterministic parser script costing **\$0.00** in LLM API tokens.

- **Escalation Path:** Only if a transaction ledger flags a cross-border wire, SWIFT messaging format anomalies, or multi-depositor accounts over **\$10,000** does the pre-router escalate and initialize the full **LangGraph agent reasoning loop**.

9.3 Semantic Cache Architecture with Redis

We configure a vector database database inside Redis using a similarity threshold index to stop repetitive questions from triggering costly re-runs.

File: src/utils/cache.py

```
import redis
```

```
from sentence_transformers import SentenceTransformer
```

```
r = redis.Redis(host='localhost', port=6379, db=0)
```

```
model = SentenceTransformer('all-MiniLM-L6-v2')
```

```
def check_semantic_cache(query: str, threshold: float = 0.95):
```

```
    query_vector = model.encode(query).tolist()
```

```
    # Query Redis Vector index for previous audits within similarity threshold
```

```
    results = r.ft("idx:cache").search(query_vector)
```

```
    if results.docs and results.docs[0].score >= threshold:
```

```
        return results.docs[0].compliance_report # Serve instant cache hit
```

```
    return None
```

9.4 Context Compaction & Top-K Reranking

Passing hundreds of raw retrieved sections into our system prompts quickly consumes token allowances.

- **Implementation:** Instead of feeding all matched ChromaDB records into gpt-4o, we retrieve the top 15 results, then run them through a lightweight **FlashRank Re-ranker** model.
- **The Benefit:** We prune the context down to the absolute top 4 most highly relevant regulatory sentences, shrinking the dynamic context window size by up to **70%** and ensuring the core auditor agent receives only non-noisy, signal-dense inputs.

10. Phase 8: FastAPI Microservice & Containerization

We expose the engine using an asynchronous FastAPI router (graph.ainvoke), enabling robust

background processing. The service is packaged using a multi-stage Dockerfile:

```
# Multi-stage build for microservice speed and size
FROM python:3.11-slim AS builder
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir --user -r requirements.txt

FROM python:3.11-slim AS runner
WORKDIR /app
COPY --from=builder /root/.local /root/.local
COPY . .
ENV PATH=/root/.local/bin:$PATH
EXPOSE 8000
CMD ["uvicorn", "src.main:app", "--host", "0.0.0.0", "--port", "8000"]
```

11. Project Milestones & Weekly Checklist

Phase 1: Ingestion & Vector Ingestion Grounding

- Initialize workspace with uv and configure project folders.
- Download the **Kaggle SAML-D transactional dataset** and **ObliQA ADGM files**.
- Write `src/utils/pdf_generator.py` to convert SAML-D data into SWIFT format logs.
- Code the semantic chunking logic using cosine distance sentence embeddings.
- Load regulatory PDF compliance rules and chunked records into ChromaDB.

Phase 2: LangGraph DAG Development

- Define the global graph AgentState schema using typed dictionaries.
- Code the Extraction and AML Audit Nodes and connect them using LangGraph.
-

Implement the Auditor Critic Node with query refinement loops back to ChromaDB.

- Connect fallback routes for failed extraction parses or unexpected inputs.

Phase 3: Observability, Evaluations, & Optimization

- Integrate LangSmith telemetry tracing, configure run-tagging, and custom client metadata.
- Write the automated **DeepEval evaluation test harness** inside tests/eval_suite.py measuring Faithfulness and Relevancy.
- Implement the pre-router node for **Hierarchical Cost Routing**.
- Setup **Redis Vector Semantic Cache** and integrate re-ranking context pruning logic.

Phase 4: Frontend Cockpit & Packaging

- Enforce output schemas strictly utilizing JSON-binding schemas from Pydantic.
- Build the Streamlit Ingestion Sidebar and Active Audit Workspace dashboards.
- Implement the interactive Auditor Citations Drawer mapping sources to compliance chunks.
- Dockerize the service using a multi-stage optimized runtime build.